Postgres e SQL - Parte #01

Banco de Dados

Prof. Dr. Carlos Melo

carlos.melo@upe.br

UPE UNIVERSIDADE DE PERNAMBUCO
CAMPUS GARANHUNS

Projeto (3pts na média final)

- Divisão dos grupos (no máx 4 pessoas por grupo)
- Aplicação real para resolver algum problema da universidade, comunidade ou do curso de vocês (30%)
  - >= 20 tabelas e/ou **views** + **triggers** + **procedures**
- Projeto completo do banco de dados **normalizado** (40%)
  - Modelo conceitual, lógico e físico
- *Seeders* para popular o banco (10%)
  - *scripts* para realização das principais consultas
- Testes a nível de aplicação e.g Python/Java (10%)
- Documentação (10%)

SQLite 👋

- Usamos o SQLite. Era leve, morava na RAM ou em um arquivo .db qualquer, mas rodava direto no Python.
- É uma excelente ferramenta para aprender DDL e DML.
- Porém não foi feito para milhares de usuário acessando o sistema ao mesmo tempo.
- Tambem não tem gestão complexa de permissões.
- Vamos subir a régua e entrar no mundo dos SGBD que seguem o modelo **cliente-servidor** .

PostgreSQL 🐘

- Um dos SGBDs de código aberto mais utilizados no mundo.
- Padrão de mercado para sistemas críticos.
- Segue rigorosamente o padrão SQL.

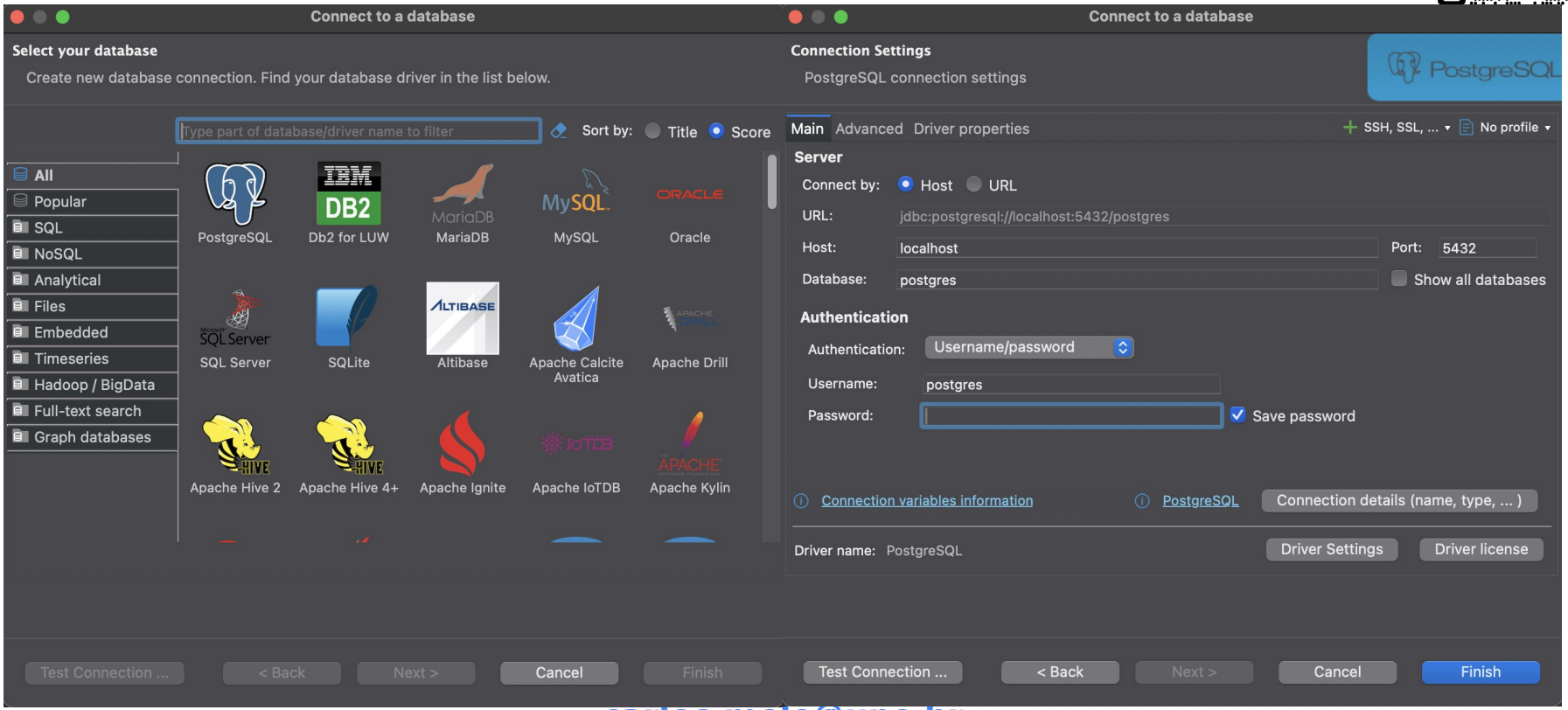
DBeaver

- O PostgreSQL não possui uma interface gráfica própria
  - é um servidor para o banco de dados
- Precisamos de um "cliente" para interagir com ele.
- O DBeaver é o cliente universal para interagir com os mais diversos servidores de banco de dados

Conectando <->

- 1) Tenha o PostgreSQL instalado em sua máquina
- 2) Tenha o DBeaver instalado em sua máquina
- Durante a instalação do postgres você define coisas como:
  - a porta padrão do serviço (geralmente 5432)
  - o hospedeiro (localhost/endereço remoto)
  - usuário e password (precisa ser definido pelo usuário)

Conectando <->


Connect to a database
Select your database
Create new database connection. Find your database driver in the list below.
Type part of database/driver name to filter
Sort by:
Title
Score
All
Popular
SQL
NoSQL
Analytical
Files
Embedded
Timeseries
Hadoop / BigData
Full-text search
Graph databases
PostgreSQL
Db2 for LUW
MariaDB
MySQL
Oracle
SQL Server
SQLite
Altibase
Apache Calcite Avatica
Apache Drill
Apache Hive 2
Apache Hive 4+
Apache Ignite
Apache IoTDB
Apache Kylin
Test Connection ...
< Back
Next >
Cancel
Finish
Connect to a database
Connection Settings
PostgreSQL connection settings
PostgreSQL
Main
Advanced
Driver properties
SSH, SSL, ...
No profile
Server
Connect by:
Host
URL
URL:
jdbc:postgresql://localhost:5432/postgres
Host:
localhost
Port:
5432
Database:
postgres
Show all databases
Authentication
Authentication:
Username/password
Username:
postgres
Password:
Save password
Connection variables information
PostgreSQL
Connection details (name, type, ...)
Driver name:
PostgreSQL
Driver Settings
Driver license
Test Connection ...
< Back
Next >
Cancel
Finish

Conectando >_ <-> 🐘

- Mas e se eu não tiver o DBeaver?
  - Terminal >_
- Para abrir o cliente em linha de comando do postgres usamos:
  - **psql -U username -d nomebanco**

```
# psql -U admin -d postgres
psql (16.13 (Debian 16.13-1.pgdg13+1))
Type "help" for help.

postgres=#
```

Criando um novo banco de dados >_

- No prompt **psql**:

```
1 CREATE DATABASE nome_do_banco;
```

- Conectando-se ao novo banco: **\c nome_do_banco**
- O prompt irá mudar para: **nome_do_banco=#**
- Também é possível criar um Esquema
  - uma "pasta" dentro do banco

```
1 CREATE SCHEMA meu_esquema;
```

Comandos essenciais no psql >_

Comando	Descrição
`\c nome_do_banco`	Conecta-se ao banco
`\l`	Lista todos os bancos de dados do servidor
`\dn`	Lista todos os esquemas do banco de dados atual
`\dt`	Lista todas as tabelas do banco atual
`\d nome_da_tabela`	Mostra a estrutura (colunas e seus tipos) de uma tabela específica
`\q`	Sai do psql e volta para o terminal padrão

Exemplo >_

- Uma vez no **psql…**
- Vamos criar a base de dados sistema_academico:
  - `CREATE DATABASE sistema_academico;`
- Para nos conectarmos ao novo banco usamos o comando **\c**
  - **\c sistema_academico**
- O prompt irá mudar para **sistema_academico=#**
- Vamos criar um novo Esquema:
  - **CREATE SCHEMA turma_2026;**

Exemplo >_

- Criaremos a tabela **aluno** dentro do schema **turma_2026**

```sql
CREATE TABLE turma_2026.aluno (
    matricula SERIAL PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE
);
```

Exemplo >_

- Populemos a tabela **aluno** dentro do schema **turma_2026**

```sql
INSERT INTO turma_2026.aluno (nome, email)
VALUES ('Severo', 'severo@email.com');
```

Exemplo >_

- E a consulta padrão

```
1 SELECT * FROM turma_2026.aluno;
```

```
teste_aula=# SELECT * FROM turma_2026.aluno;
 matricula |  nome   |       email
-----------+---------+------------------
         1 | A       | B
         2 | SEVERO  | SEVERO@EMAIL.COM
(2 rows)

teste_aula=#
```

Exercício

- Crie a tabela professor.
  - Ela deve ter um cpf (varchar, Chave Primária) e um nome (varchar, obrigatório).
- Insira dois novos professores na tabela professor
- Escreva um SELECT que mostre somente o nome dos professores

Construindo a Empresa no Postgres

- O Script está disponível no github da disciplina

Anatomia do **SELECT**

- O **SELECT** é a ferramenta mais poderosa em um BD Relacional.
- Ele não altera nada, apenas projeta os dados na tela.
- O asterisco traz tudo, mas em sistemas grandes pode elevar o processamento, por isso:

```sql
SELECT coluna1, coluna2
FROM tabela
WHERE condicao
ORDER BY coluna1;
```

Filtros com **WHERE** e operadores lógicos

- Se o **SELECT** escolhe as colunas, o **WHERE** escolhe as linhas
- Operadores de Comparação: **=, >, <, >=, <=, <>** (diferente)
- Operadores Lógicos: **AND** (e), **OR** (ou), **NOT** (não)
- Quero o nome e o salário de quem ganha **mais de 5000** e trabalha no **departamento 1**.

```sql
SELECT nome, salario
FROM empregado
WHERE salario > 5000 AND numero_departamento = 1;
```

O operador **LIKE** para buscar padrões

- O = exige que a palavra ou valor seja exatamente igual.
- Mas se quisermos procurar por um empregado que não sabemos seu nome completo?
- Nesse caso usamos o **LIKE**
  - **%:** substitui qualquer quantidade de caracteres
  - **_:** substitui exatamente um caractere
- Encontrar qualquer pessoa que tenha "Silva" no nome:

```sql
SELECT nome, cpf
FROM empregado
WHERE nome LIKE '%Silva%';
```

IN e BETWEEN

- Como buscar quem trabalha no
  - **departamento 1 OU no departamento 3**?

```sql
SELECT nome
FROM empregado
WHERE numero_departamento = 1 OR numero_departamento = 3;
```

```sql
SELECT nome
FROM empregado
WHERE numero_departamento IN (1, 3);
```

Ordenando com **ORDER BY**

- Pordemos ordenar a saída do **SELECT** por meio do **ORDER BY**
  - **ASC**: Crescente (A-Z, 0-9) - É o default
  - **DESC**: Decrescente (Z-A, 9-0)
- Listando empregados do maior salário para o menor:

```
SELECT nome, salario
FROM empregado
ORDER BY salario DESC;
```

Modificando dados com UPDATE e SET

- Dados podem ser alterados constantemente: salários podem ser reajustados, pessoas podem mudar de departamento ou serem desligadas.

> NUNCA FAÇA UPDATE SEM WHERE

- O João foi promovido e o salário dele é 6000.

```
UPDATE tabela
SET coluna1 = valor1, coluna2 = valor2
WHERE condicao;
```

```
UPDATE empregado
SET salario = 6000.00
WHERE cpf = '111.111.111-11';
```

Removendo dados com **DELETE**

- Quando um registro não é mais necessário ele pode ser removido

> NUNCA FAÇA DELETE SEM WHERE

- Removendo a alocação de Fernando do projeto 30

```sql
DELETE FROM tabela
WHERE condicao;
```

```sql
DELETE FROM empregado_projeto
WHERE cpf_empregado = '555.555.555-55' AND codigo_projeto = 30;
```

Exercícios

1. O diretor precisa de um relatório com o nome e o salário de todos os empregados da empresa. O relatório deve estar ordenado do maior para o menor salário.
2. Liste o codigo e o nome de todos os projetos que a empresa possui atualmente.
3. O RH solicitou uma lista com os dados de todos os empregados que nasceram antes de 1990.
4. Vá a tabela empregado_projeto e descubra os CPFs que trabalharam entre 20 e 30 horas em seus respectivos projetos.

Exercícios

1. A diretoria quer saber quais projetos estão rodando em São Paulo ou em Recife.
2. Houve uma mudança de planos e o projeto ‘App Mobile’ foi transferido para o Rio de Janeiro. Escreva o comando para atualizar essa localização no banco de dados.

Prof. Dr. Carlos Melo
https://calendly.com/prof-casm
carlos.melo@upe.br

UPE UNIVERSIDADE DE PERNAMBUCO CAMPUS GARANHUNS